

“Universidad Autónoma de Zacatecas”

Francisco Garcia Salinas

Programa: Ingeniería de software



Universidad Autónoma de Zacatecas



Tema: Proyecto Final

“Ventas App”

Materia: Introducción a la programación

Docente: Santiago Esparza Guererro

Título del proyecto: Ventas APP

(líder) Alan Alfonso Contreras Montalgo - matrícula = 42204063

Joxan Eliu Solis Rodriguez - matrícula = 42204107

Gael Alvarado Zapata - matrícula = 43421248

2. DESCRIPCIÓN DEL PROGRAMA

¿Qué hace el programa?

VentasApp es una solución web robusta diseñada para la administración integral del ciclo comercial en tiempo real. El sistema unifica operaciones clave mediante los siguientes componentes:

- **Registro transaccional inmediato:** Captura estructurada de datos de venta, vinculando ID del producto, cantidad, precio pactado y metadatos del vendedor activo.
- **Gestión dinámica de inventario:** Sincronización automatizada del stock disponible; cada transacción genera un decremento atómico en base de datos.
- **Seguimiento y control de operadores:** Delimitación de funciones mediante control de acceso basado en roles (Administrador y Vendedor).
- **Subsistema de reporte dinámico:** Herramientas integradas para exportación estructurada de datos en formato CSV y generación automática de métricas operativas clave.
- **Trazabilidad y auditoría (Log de Actividad):** Registro inmutable de operaciones críticas para la identificación de responsabilidades y detección de anomalías.
- **Seguridad de sesión:** Mecanismos de autenticación persistente y segura mediante manejo de cookies de sesión cifradas.

¿Cuál es su finalidad?

La arquitectura de la aplicación tiene como objetivo principal la automatización y centralización de la lógica de negocio comercial de la organización, promoviendo:

1. **Transparencia operativa:** Visualización inmediata del desempeño transaccional sin desfases temporales.
2. **Gobernanza administrativa:** Herramientas de supervisión exhaustiva sobre el catálogo de productos, cuentas de usuarios y flujos de caja.
3. **Eficiencia de procesos:** Mitigación del error humano asociado a la transcripción de registros y reducción de tiempos muertos operativos.
4. **Inteligencia de negocio básica:** Acceso simplificado a métricas analíticas (ingresos mensuales, ticket promedio y productos líderes).
5. **Cumplimiento técnico de control:** Disponibilidad de registros estructurados para auditorías internas o revisiones contables.

¿Qué problema busca resolver?

Contexto del Escenario Manual

El modelo de gestión previo de la empresa (enfocada en tecnología y accesorios) dependía enteramente de flujos de trabajo tradicionales no integrados, lo que manifestaba los siguientes síntomas críticos:

- **Silos de información:** Registros operados en archivos aislados de hojas de cálculo

- (Excel) distribuidos de forma local en los equipos de cada vendedor.
- **Corrupción e inconsistencia formal:** Ausencia de validaciones de tipo en la captura de datos, provocando formatos de fecha heterogéneos y discrepancias ortográficas en nombres de productos.
 - **Desfase crítico de almacén:** Inexistencia de comunicación en tiempo real entre el punto de venta y el almacén, derivando de forma frecuente en sobreventa de artículos (quiebres de stock).
 - **Vulnerabilidad de seguridad y anonimato:** Incapacidad de rastrear modificaciones, cancelaciones o inserciones sospechosas de transacciones por falta de firmas de usuario.
 - **Costos de consolidación:** Esfuerzo administrativo intensivo y propenso a errores al final de cada periodo comercial para estructurar reportes unificados.

La Solución Propuesta: VentasApp

El despliegue del sistema resuelve de raíz estas ineficiencias mediante un núcleo tecnológico integrado:

- **Única fuente de verdad:** Base de datos centralizada y normalizada bajo el motor SQLite3.
- **Sincronización atómica de stock:** Restricciones lógicas que impiden transacciones si el stock es menor a la demanda solicitada.
- **Capa de auditoría obligatoria:** Interceptores que graban de forma transparente metadatos del usuario (incluyendo IP) en cada mutación de datos.
- **Experiencia de usuario optimizada:** Formularios intuitivos validados tanto en el cliente (JS) como en el servidor (Flask).

3. ESPECIFICACIÓN FORMAL DEL PROBLEMA

A continuación se detallan los parámetros y restricciones que gobiernan el núcleo transaccional del sistema:

Dimensión	Componentes y Restricciones de Especificación
Entradas del Sistema	● Datos de Sesión: ID de usuario, rol asignado, IP de origen del cliente. ● Identificador de Producto: ID único entero (Llave Primaria). ● Monto de Demanda: Cantidad entera

Dimensión	Componentes y Restricciones de Especificación
	positiva (Cantidad > 0). ● Precio de Venta: Valor flotante con precisión de dos decimales (Precio > 0). ● Marcas Temporales: Fecha y hora en formato estándar ISO/SQLite (YYYY-MM-DD HH:MM:SS). ● Metadata Adicional: Cadena de texto libre para notas aclaratorias (Opcional).
Lógica del Proceso	1. Validación de Sesión: El motor verifica la vigencia de la cookie a través del gestor de contexto de Flask-Login. 2. Evaluación de Existencia: Consulta al catálogo para validar el ID del producto y verificar que se encuentre con estado activo (activo = 1). 3. Control de Disponibilidad: Verificación condicional: Si stock_actual >= cantidad_solicitada continúa; en caso contrario, interrumpe el flujo. 4. Cómputo Financiero: Operación aritmética de costo total: total = round(precio_unitario * cantidad, 2). 5. Inserción Transaccional: Registro persistente en la tabla ventas. 6. Actualización de Inventario: Mutación atómica del stock en la tabla productos mediante decremento. 7. Escritura en Log: Llamada imperativa a la función interna de auditoría para asentar la acción. 8. Confirmación: Retorno de estructura serializada JSON con código de estado HTTP 200 y el ID de la venta.
Salidas del Sistema	● Identificador único de venta generado de forma autoincremental en la base de

Dimensión	Componentes y Restricciones de Especificación
	datos. • Mensajes de error estructurados con códigos HTTP semánticos (400 Bad Request, 403 Forbidden, 404 Not Found, 500 Internal Error). • Estado físico de almacén actualizado inmediatamente en todas las interfaces de consulta. • Registro inmutable del evento agregado a las vistas del panel de administración.
Capacidades Extendidas	• Edición controlada de transacciones históricas con recalibración automática de inventarios anteriores. • Cancelación lógica de ventas con políticas de reversión total de mercancías al inventario. • Generación de vistas optimizadas para impresión física de comprobantes de pago (Tickets). • Mecanismos automáticos de alerta visual cuando el stock desciende del umbral de seguridad ($\text{stock} \leq \text{stock_minimo}$).

4. HERRAMIENTAS UTILIZADAS Y ENTORNO

Tecnologías de Desarrollo (Stack)

Herramienta	Tipo	Versión Mínima	Propósito Específico en la Solución
Python	Lenguaje Backend	3.8+	Soporte global del core del software, tipado dinámico y manejo de estructuras.
Flask	Framework Web WSGI	3.0.0	Enrutamiento de peticiones HTTP, manejo de contextos de solicitud y renderizado de plantillas.
Flask-Login	Extensión de Sesiones	0.6.3	Mapeo automatizado de usuarios en cookies de sesión seguras y protección de endpoints por decoradores.
SQLite3	RDBMS Embebido	3.x	Motor de persistencia local con aislamiento transaccional y activación de modo Write-Ahead Logging (WAL).
HTML5 / CSS3	Marcado y Estilos	Estándar W3C	Estructuración semántica del frontend y

Herramienta	Tipo	Versión Mínima	Propósito Específico en la Solución
			maquetación visual adaptativa de las vistas.
JavaScript	Scripting Frontend	ECMAScript 6	Consumo asíncrono de APIs (Fetch API), manipulación del DOM y validaciones en tiempo de cliente.

Entorno y Dependencias de Producción

El control de versiones del entorno se rige por un archivo formal de dependencias estándar (requirements.txt):

```
flask==3.0.0          # Framework de enrutamiento principal
flask-login==0.6.3    # Administración del ciclo de vida del usuario
werkzeug==3.0.1       # Herramientas de seguridad criptográfica (SHA-256
para contraseñas)
```

Para la orquestación del proyecto se empleó la suite integrada de **Visual Studio Code** como IDE base, utilizando ramas de control con **Git**, y validaciones funcionales directas sobre terminales híbridas PowerShell y entornos de inspección web en navegadores modernos Chromium.

Uso Estratégico de Inteligencia Artificial en Ingeniería de Software:

Durante el ciclo de desarrollo de VentasApp, se utilizaron asistentes de IA con un enfoque metodológico orientado a: la aceleración en la maquetación de esqueletos de código HTML/CSS, la optimización de sentencias SQL complejas (reducción de subconsultas costosas), el análisis estructural para debugging asíncrono y el diseño automatizado de las matrices de pruebas unitarias.

5. ARQUITECTURA Y CÓDIGO FUENTE

Estructura del Proyecto

```
sales_system/
├─ app.py          # Orquestador principal, definición de rutas y API
REST
├─ auth.py         # Módulo encargado de la capa de seguridad, modelos y
roles
├─ database.py     # Inicialización del RDBMS, inyección de semillas y
utilidades SQL
├─ requirements.txt # Declaración explícita de bibliotecas de producción
├─ sales.db        # Archivo binario de base de datos SQLite (Generada
en Runtime)
├─ static/         # Recursos estáticos del sistema (Hojas de estilo
CSS, scripts JS)
├─ templates/      # Repositorio de plantillas HTML compiladas por
Jinja2
  ├─ login.html    # Acceso y control de seguridad inicial
  ├─ dashboard.html # Tablero principal de analíticas agregadas
  ├─ historial.html # Grid de visualización transaccional
  ├─ nueva_venta.html # Captura y validación de nuevas operaciones
  ├─ editar_venta.html # Modificación asistida con cómputo de almacén
  ├─ ticket.html    # Plantilla limpia preparada para impresión térmica
  └─ productos.html # Catálogo maestro de artículos
```

Código Fuente - Archivos Centrales de la Aplicación

Módulo 1: app.py (Controlador de Rutas y Lógica Web)

```
"""
Aplicación principal de VentasApp – Sistema de Registro de Ventas.
Contiene todas las rutas Flask del sistema.
"""

import csv
import io
import sqlite3
from datetime import datetime, timedelta

from flask import (Flask, redirect, render_template, request,
                   jsonify, url_for, session, make_response)
from flask_login import login_required, login_user, logout_user, current_user
from werkzeug.security import generate_password_hash, check_password_hash

import auth
import database

# — Inicialización de la aplicación —
```



```

app = Flask(__name__)
app.secret_key = "ventasapp_uaz_ingenieria_software_2025_secret"
app.config["REMEMBER_COOKIE_DURATION"] = 28800 # 8 horas

auth.login_manager.init_app(app)
auth.login_manager.login_view = "login"

def get_ip():
    """Obtiene la dirección IP real del cliente."""
    return request.headers.get("X-Forwarded-For", request.remote_addr)

# -----
# AUTH
# -----

@app.route("/")
def index():
    """Redirecciona al dashboard si está autenticado, de lo contrario al login."""
    if current_user.is_authenticated:
        return redirect(url_for("dashboard"))
    return redirect(url_for("login"))

@app.route("/login", methods=["GET", "POST"])
def login():
    """
    Maneja el inicio de sesión de usuarios.
    GET: muestra el formulario.
    POST: valida credenciales y establece la sesión.
    """
    if current_user.is_authenticated:
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        data = request.get_json(silent=True) or {}
        username = str(data.get("username", "")).strip()
        password = str(data.get("password", ""))

        if not username or not password:
            return jsonify({"error": "Usuario y contraseña son requeridos"}),
400

        usuario, mensaje = auth.autenticar_usuario(username, password, get_ip())
        if not usuario:
            return jsonify({"error": mensaje}), 401

        login_user(usuario, remember=True)
        return jsonify({"success": True, "redirect": url_for("dashboard")})

```

```

    return render_template("login.html")

@app.route("/logout")
@login_required
def logout():
    """Cierra la sesión del usuario y registra la actividad."""
    conn = database.get_connection()
    try:
        database.registrar_actividad(
            conn, current_user.id, current_user.nombre,
            "logout", "Cierre de sesión", get_ip()
        )
        conn.commit()
    finally:
        conn.close()
    logout_user()
    return redirect(url_for("login"))

# -----
# DASHBOARD
# -----

@app.route("/dashboard")
@login_required
def dashboard():
    """Muestra el panel principal con métricas, alertas y ventas recientes."""
    metricas = database.get_dashboard_metrics(current_user.id, current_user.rol)
    return render_template("dashboard.html", **metricas)

# -----
# VENTAS - CRUD Y OPERACIONES
# -----

@app.route("/ventas")
@login_required
def historial():
    """Renderiza la página del historial de ventas."""
    return render_template("historial.html")

@app.route("/api/ventas")
@login_required
def api_ventas():
    """Devuelve la lista de ventas en JSON con filtros opcionales."""
    conn = database.get_connection()
    try:
        condiciones = ["1=1"]

```

```

params = []

if current_user.rol != "admin":
    condiciones.append("vendedor_id=?")
    params.append(current_user.id)

fecha_filtro = request.args.get("fecha", "")
hoy = datetime.now().strftime("%Y-%m-%d")
if fecha_filtro == "hoy":
    condiciones.append("DATE(fecha)=?")
    params.append(hoy)
elif fecha_filtro == "semana":
    condiciones.append("DATE(fecha) >= DATE('now', '-7 days')")
elif fecha_filtro == "mes":
    condiciones.append("DATE(fecha) >= DATE('now', '-30 days')")
elif "," in fecha_filtro:
    partes = fecha_filtro.split(",")
    condiciones.append("DATE(fecha) BETWEEN ? AND ?")
    params.extend([partes[0], partes[1]])

cat = request.args.get("categoria", "")
if cat:
    condiciones.append("categoria=?")
    params.append(cat)

if current_user.rol == "admin":
    vend = request.args.get("vendedor", "")
    if vend:
        condiciones.append("vendedor_nombre LIKE ?")
        params.append(f"%{vend}%")

estado = request.args.get("estado", "")
if estado in ("completada", "cancelada"):
    condiciones.append("estado=?")
    params.append(estado)

busqueda = request.args.get("busqueda", "").strip()
if busqueda:
    condiciones.append("(producto_nombre LIKE ? OR notas LIKE ?)")
    params.extend([f"%{busqueda}%", f"%{busqueda}%"])

where = " AND ".join(condiciones)

orden_map = {
    "reciente": "fecha DESC",
    "antigua": "fecha ASC",
    "mayor": "total DESC",
    "menor": "total ASC",
}

orden = orden_map.get(request.args.get("orden", "reciente"), "fecha DESC")

```

```

        pagina = max(1, int(request.args.get("pagina", 1)))
        por_pag = 50
        offset = (pagina - 1) * por_pag

        total_row = conn.execute(
            f"SELECT COUNT(*) as cnt FROM ventas WHERE {where}", params
        ).fetchone()
        total = total_row["cnt"]

        rows = conn.execute(
            f"SELECT v.*, p.emoji FROM ventas v "
            f"LEFT JOIN productos p ON v.producto_id=p.id "
            f"WHERE {where} ORDER BY {orden} LIMIT {por_pag} OFFSET {offset}",
            params
        ).fetchall()

        return jsonify({
            "ventas": [dict(r) for r in rows],
            "total": total,
            "pagina": pagina,
        })
    finally:
        conn.close()

```

```

@app.route("/ventas/nueva", methods=["GET", "POST"])
@login_required
def nueva_venta():
    """Procesa el registro de la venta, reduce stock y registra actividad."""
    if request.method == "GET":
        return render_template("nueva_venta.html")

    data = request.get_json(silent=True) or {}
    conn = database.get_connection()
    try:
        producto_id = int(data.get("producto_id", 0))
        cantidad = int(data.get("cantidad", 0))
        precio_unitario = float(data.get("precio_unitario", 0))
        notas = str(data.get("notas", "")).strip()
        fecha = str(data.get("fecha",
datetime.now().strftime("%Y-%m-%d %H:%M:%S")))

        if producto_id <= 0 or cantidad <= 0 or precio_unitario <= 0:
            return jsonify({"error": "Datos incompletos o inválidos"}), 400

        prod = conn.execute(
            "SELECT * FROM productos WHERE id=? AND activo=1", (producto_id,)
        ).fetchone()
        if not prod:
            return jsonify({"error": "Producto no encontrado"}), 404

```

```

        if prod["stock"] < cantidad:
            return jsonify({"error": f"Stock insuficiente. Disponible: {prod['stock']}"}), 400

    total = round(precio_unitario * cantidad, 2)

    cur = conn.execute(
        """INSERT INTO ventas
            (producto_id, producto_nombre, categoria, cantidad,
precio_unitario,
            total, vendedor_id, vendedor_nombre, fecha, notas)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (producto_id, prod["nombre"], prod["categoria"],
        cantidad, precio_unitario, total,
        current_user.id, current_user.nombre, fecha, notas or None)
    )
    venta_id = cur.lastrowid

    conn.execute(
        "UPDATE productos SET stock = stock - ? WHERE id=?",
        (cantidad, producto_id)
    )

    database.registrar_actividad(
        conn, current_user.id, current_user.nombre, "crear",
        f"Nueva venta #{venta_id}: {prod['nombre']} x{cantidad} =
${total:,.2f}",
        get_ip()
    )
    conn.commit()
    return jsonify({"success": True, "venta_id": venta_id, "mensaje": "Venta
registrada exitosamente"})

except (ValueError, TypeError) as e:
    conn.rollback()
    return jsonify({"error": "Datos inválidos: " + str(e)}), 400
except sqlite3.Error:
    conn.rollback()
    return jsonify({"error": "Error en la base de datos"}), 500
finally:
    conn.close()

@app.route("/ventas//editar", methods=["GET", "PUT"])
@login_required
def editar_venta(vid):
    """Actualiza los datos de la venta y ajusta el stock de forma
correspondiente."""
    conn = database.get_connection()
    try:
        venta = conn.execute(

```

```

        "SELECT v.*, p.emoji FROM ventas v LEFT JOIN productos p ON
v.producto_id=p.id WHERE v.id=?",
        (vid,)
    ).fetchone()
    if not venta:
        conn.close()
    if request.method == "GET":
        return render_template("editar_venta.html", venta_id=vid,
error="Venta no encontrada")
    return jsonify({"error": "Venta no encontrada"}), 404

    puede_editar = (current_user.rol == "admin" or venta["vendedor_id"] ==
current_user.id)
    if not puede_editar:
        conn.close()
        return jsonify({"error": "No tienes permisos para editar esta
venta"}), 403

    if request.method == "GET":
        conn.close()
        return render_template("editar_venta.html", venta_id=vid)

    data = request.get_json(silent=True) or {}
    nueva_cantidad = int(data.get("cantidad", venta["cantidad"]))
    nuevo_precio = float(data.get("precio_unitario",
venta["precio_unitario"]))
    nuevas_notas = str(data.get("notas", venta["notas"] or "")).strip()
    nueva_fecha = str(data.get("fecha", venta["fecha"]))
    nuevo_producto_id = int(data.get("producto_id", venta["producto_id"] or
0))

    if nueva_cantidad <= 0 or nuevo_precio <= 0:
        return jsonify({"error": "Cantidad y precio deben ser mayores a
cero"}), 400

    if nuevo_producto_id and nuevo_producto_id != venta["producto_id"]:
        if venta["producto_id"]:
            conn.execute("UPDATE productos SET stock=stock+? WHERE id=?",
                (venta["cantidad"], venta["producto_id"]))
            prod_nuevo = conn.execute(
                "SELECT * FROM productos WHERE id=? AND activo=1",
(nuevo_producto_id,)
            ).fetchone()
            if not prod_nuevo or prod_nuevo["stock"] < nueva_cantidad:
                return jsonify({"error": "Stock insuficiente en el nuevo
producto"}), 400
            conn.execute("UPDATE productos SET stock=stock-? WHERE id=?",
                (nueva_cantidad, nuevo_producto_id))
            prod_nombre = prod_nuevo["nombre"]
            prod_cat = prod_nuevo["categoria"]
        else:

```

```

        diff = nueva_cantidad - venta["cantidad"]
        if diff > 0:
            prod_actual = conn.execute(
                "SELECT stock FROM productos WHERE id=?",
                (venta["producto_id"],)
            ).fetchone()
            if prod_actual and prod_actual["stock"] < diff:
                return jsonify({"error": "Stock insuficiente"}), 400
            conn.execute("UPDATE productos SET stock=stock-? WHERE id=?",
                (diff, venta["producto_id"]))
            prod_nombre = venta["producto_nombre"]
            prod_cat = venta["categoria"]
            nuevo_producto_id = venta["producto_id"]

        nuevo_total = round(nueva_cantidad * nuevo_precio, 2)

        conn.execute(
            """UPDATE ventas SET producto_id=?, producto_nombre=?, categoria=?,
                cantidad=?, precio_unitario=?, total=?, notas=?, fecha=?,
                editada=1, editada_por=?, editada_en=CURRENT_TIMESTAMP WHERE
id=?""",
            (nuevo_producto_id, prod_nombre, prod_cat,
                nueva_cantidad, nuevo_precio, nuevo_total,
                nuevas_notas or None, nueva_fecha,
                current_user.nombre, vid)
        )
        database.registrar_actividad(
            conn, current_user.id, current_user.nombre, "editar",
            f"Venta #{vid} editada por {current_user.nombre}", get_ip()
        )
        conn.commit()
        return jsonify({"success": True, "mensaje": "Venta actualizada
correctamente"})

    except (ValueError, TypeError):
        conn.rollback()
        return jsonify({"error": "Datos inválidos"}), 400
    except sqlite3.Error:
        conn.rollback()
        return jsonify({"error": "Error en la base de datos"}), 500
    finally:
        conn.close()

```

```

@app.route("/ventas//cancelar", methods=["PUT"])
@login_required
def cancelar_venta(vid):
    """Cancela una venta existente de forma lógica y restaura el stock."""
    conn = database.get_connection()
    try:
        venta = conn.execute("SELECT * FROM ventas WHERE id=?",

```

```

(vid,)).fetchone()
    if not venta:
        return jsonify({"error": "Venta no encontrada"}), 404
    if venta["estado"] == "cancelada":
        return jsonify({"error": "La venta ya está cancelada"}), 400

    puede = (current_user.rol == "admin" or venta["vendedor_id"] ==
current_user.id)
    if not puede:
        return jsonify({"error": "No tienes permisos para cancelar esta
venta"}), 403

    data = request.get_json(silent=True) or {}
    motivo = str(data.get("motivo", "")).strip()
    if not motivo:
        return jsonify({"error": "El motivo de cancelación es requerido"}),
400

    conn.execute(
        "UPDATE ventas SET estado='cancelada', motivo_cancelacion=? WHERE
id=?",
        (motivo, vid)
    )
    if venta["producto_id"]:
        conn.execute(
            "UPDATE productos SET stock=stock+? WHERE id=?",
            (venta["cantidad"], venta["producto_id"])
        )
    database.registrar_actividad(
        conn, current_user.id, current_user.nombre, "cancelar",
        f"Venta #{vid} cancelada: {motivo}", get_ip()
    )
    conn.commit()
    return jsonify({"success": True, "mensaje": "Venta cancelada
exitosamente"})

except sqlite3.Error:
    conn.rollback()
    return jsonify({"error": "Error en la base de datos"}), 500
finally:
    conn.close()

@app.route("/ventas/exportar")
@login_required
def exportar_ventas():
    """Genera y retorna un stream CSV estructurado con las ventas del usuario
actual."""
    conn = database.get_connection()
    try:
        if current_user.rol == "admin":

```



```

        rows = conn.execute(
            "SELECT id, producto_nombre, categoria, cantidad,
precio_unitario, "
            "total, vendedor_nombre, estado, notas, fecha FROM ventas ORDER
BY fecha DESC"
        ).fetchall()
    else:
        rows = conn.execute(
            "SELECT id, producto_nombre, categoria, cantidad,
precio_unitario, "
            "total, vendedor_nombre, estado, notas, fecha FROM ventas "
            "WHERE vendedor_id=? ORDER BY fecha DESC", (current_user.id,)
        ).fetchall()

```

```

    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["ID", "Producto", "Categoría", "Cantidad", "Precio
Unitario",
                    "Total", "Vendedor", "Estado", "Notas", "Fecha"])
    for r in rows:
        writer.writerow([
            r["id"], r["producto_nombre"], r["categoria"],
            r["cantidad"], f"${r['precio_unitario']:.2f}",
            f"${r['total']:.2f}", r["vendedor_nombre"],
            r["estado"], r["notas"] or "", r["fecha"]
        ])

```

```

    output.seek(0)
    resp = make_response(output.getvalue())
    resp.headers["Content-Disposition"] = "attachment;
filename=ventas_export.csv"
    resp.headers["Content-Type"] = "text/csv; charset=utf-8"
    return resp
finally:
    conn.close()

```

```

@app.route("/productos", methods=["GET"])
@login_required
def productos():
    """Renderiza el catálogo maestro de artículos."""
    return render_template("productos.html")

```

```

@app.route("/api/productos", methods=["GET"])
@login_required
def api_productos():
    """Devuelve JSON de productos activos con sus métricas calculadas."""
    conn = database.get_connection()
    try:
        rows = conn.execute(

```

```

        """SELECT p.*,
            COALESCE((SELECT COUNT(*) FROM ventas WHERE producto_id=p.id AND
estado='completada'),0) as veces_vendido
            FROM productos p WHERE p.activo=1 ORDER BY p.nombre"""
    ).fetchall()
    return jsonify([dict(r) for r in rows])
finally:
    conn.close()

```

```

@app.route("/api/productos/nuevo", methods=["POST"])
@login_required
@auth.admin_required
def nuevo_producto():
    """Crea un nuevo ítem en el catálogo de productos."""
    data = request.get_json(silent=True) or {}
    conn = database.get_connection()
    try:
        nombre = str(data.get("nombre", "")).strip()
        categoria = str(data.get("categoria", "")).strip()
        precio_base = float(data.get("precio_base", 0))
        emoji = str(data.get("emoji", "📦")).strip()
        stock = int(data.get("stock", 0))
        stock_minimo = int(data.get("stock_minimo", 5))

        if not nombre or not categoria or precio_base <= 0:
            return jsonify({"error": "Nombre, categoría y precio son
requeridos"}), 400

        cur = conn.execute(
            "INSERT INTO productos (nombre, categoria, precio_base, emoji,
stock, stock_minimo) VALUES (?, ?, ?, ?, ?, ?)",
            (nombre, categoria, precio_base, emoji, stock, stock_minimo)
        )
        prod_id = cur.lastrowid
        database.registrar_actividad(
            conn, current_user.id, current_user.nombre, "crear",
            f"Nuevo producto creado: {nombre} (ID:{prod_id})", get_ip()
        )
        conn.commit()
        return jsonify({"success": True, "producto_id": prod_id, "mensaje":
"Producto creado exitosamente"})

    except (ValueError, TypeError):
        conn.rollback()
        return jsonify({"error": "Datos inválidos"}), 400
    except sqlite3.Error:
        conn.rollback()
        return jsonify({"error": "Error en la base de datos"}), 500
    finally:
        conn.close()

```

```

@app.route("/api/productos/", methods=["PUT"])
@login_required
@auth.admin_required
def actualizar_producto(pid):
    """Actualiza de forma completa los campos de un producto."""
    data = request.get_json(silent=True) or {}
    conn = database.get_connection()
    try:
        prod = conn.execute("SELECT * FROM productos WHERE id=?",
(pid,)).fetchone()
        if not prod:
            return jsonify({"error": "Producto no encontrado"}), 404

        nombre = str(data.get("nombre", prod["nombre"])).strip()
        categoria = str(data.get("categoria", prod["categoria"])).strip()
        precio_base = float(data.get("precio_base", prod["precio_base"]))
        emoji = str(data.get("emoji", prod["emoji"])).strip()
        stock_minimo = int(data.get("stock_minimo", prod["stock_minimo"]))
        activo = int(data.get("activo", prod["activo"]))

        conn.execute(
            "UPDATE productos SET nombre=?, categoria=?, precio_base=?, emoji=?,
stock_minimo=?, activo=? WHERE id=?",
            (nombre, categoria, precio_base, emoji, stock_minimo, activo, pid)
        )
        database.registrar_actividad(
            conn, current_user.id, current_user.nombre, "editar",
            f"Producto actualizado: {nombre} (ID:{pid})", get_ip()
        )
        conn.commit()
        return jsonify({"success": True, "mensaje": "Producto actualizado"})

    except (ValueError, TypeError):
        conn.rollback()
        return jsonify({"error": "Datos inválidos"}), 400
    except sqlite3.Error:
        conn.rollback()
        return jsonify({"error": "Error en la base de datos"}), 500
    finally:
        conn.close()

```

Módulo 2: auth.py (Capa de Autenticación de Usuarios)

```

"""
Módulo de autenticación para VentasApp.
Gestiona login, logout, sesiones y decoradores de acceso por rol.
"""

```

```

from functools import wraps
from flask import session, redirect, url_for, request, jsonify
from flask_login import LoginManager, UserMixin, login_user, logout_user,
current_user
from werkzeug.security import check_password_hash
import database

login_manager = LoginManager()

class Usuario(UserMixin):
    """Clase modelo de usuario mapeada para el control de Flask-Login."""

    def __init__(self, row):
        self.id = row["id"]
        self.nombre = row["nombre"]
        self.username = row["username"]
        self.rol = row["rol"]
        self.activo = row["activo"]
        self.avatar_emoji = row["avatar_emoji"]

    def is_active(self):
        return bool(self.activo)

    def is_admin(self):
        return self.rol == "admin"

@login_manager.user_loader
def cargar_usuario(user_id):
    """Instancia el modelo mapeado del usuario recuperado desde la BD."""
    conn = database.get_connection()
    try:
        row = conn.execute(
            "SELECT * FROM usuarios WHERE id=? AND activo=1", (int(user_id),)
        ).fetchone()
        return Usuario(row) if row else None
    finally:
        conn.close()

@login_manager.unauthorized_handler
def no_autorizado():
    return redirect(url_for("login"))

def admin_required(f):
    """Intercepta peticiones e impide el paso si el rol es diferente a
    administrador."""
    @wraps(f)
    def decorado(*args, **kwargs):

```

```

        if not current_user.is_authenticated or not current_user.is_admin():
            return jsonify({"error": "No tienes permisos para esta acción"}),
403

        return f(*args, **kwargs)

    return decorado

def autenticar_usuario(username, password, ip=""):
    """Valida las credenciales ingresadas aplicando hashing seguro."""
    conn = database.get_connection()
    try:
        row = conn.execute(
            "SELECT * FROM usuarios WHERE username=?", (username.strip(),)
        ).fetchone()

        if not row:
            database.registrar_actividad(
                conn, None, username, "login_fallido",
                f"Intento de login con usuario inexistente: {username}", ip
            )
            conn.commit()
            return None, "Usuario o contraseña incorrectos"

        if not row["activo"]:
            return None, "Cuenta desactivada. Contacta al administrador."

        if not check_password_hash(row["password_hash"], password):
            database.registrar_actividad(
                conn, row["id"], row["nombre"], "login_fallido",
                f"Contraseña incorrecta para: {username}", ip
            )
            conn.commit()
            return None, "Usuario o contraseña incorrectos"

        conn.execute(
            "UPDATE usuarios SET last_login=CURRENT_TIMESTAMP WHERE id=?",
            (row["id"],)
        )
        database.registrar_actividad(
            conn, row["id"], row["nombre"], "login",
            f"Inicio de sesión exitoso desde {ip}", ip
        )
        conn.commit()
        return Usuario(row), ""

    finally:
        conn.close()

```

Módulo 3: database.py (Abstracción de Datos y SQL)

```

"""
Módulo de base de datos para VentasApp.
Gestiona la inicialización del esquema, datos de prueba y funciones auxiliares.
"""

import sqlite3
import os
from datetime import datetime, timedelta
import random
from werkzeug.security import generate_password_hash

DB_PATH = os.path.join(os.path.dirname(__file__), "sales.db")

def get_connection():
    """Establece conexión nativa SQLite forzando llaves foráneas y modo WAL."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("PRAGMA foreign_keys=ON")
    return conn

def init_db():
    """Genera las tablas estructurales en la BD si no se detecta existencia
    previa."""
    conn = get_connection()
    try:
        cur = conn.cursor()

        cur.execute("""
            CREATE TABLE IF NOT EXISTS usuarios (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                username TEXT UNIQUE NOT NULL,
                password_hash TEXT NOT NULL,
                rol TEXT DEFAULT 'vendedor',
                activo INTEGER DEFAULT 1,
                avatar_emoji TEXT DEFAULT '👤',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                last_login TIMESTAMP
            )
        """)

        cur.execute("""
            CREATE TABLE IF NOT EXISTS productos (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                categoria TEXT NOT NULL,
                precio_base REAL DEFAULT 0,
                emoji TEXT DEFAULT '📦',
            )
        """)
    
```

```

        stock INTEGER DEFAULT 0,
        stock_minimo INTEGER DEFAULT 5,
        activo INTEGER DEFAULT 1,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """)

cur.execute("""
    CREATE TABLE IF NOT EXISTS ventas (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        producto_id INTEGER,
        producto_nombre TEXT NOT NULL,
        categoria TEXT NOT NULL,
        cantidad INTEGER NOT NULL,
        precio_unitario REAL NOT NULL,
        total REAL NOT NULL,
        vendedor_id INTEGER NOT NULL,
        vendedor_nombre TEXT NOT NULL,
        estado TEXT DEFAULT 'completada',
        motivo_cancelacion TEXT,
        notas TEXT,
        fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        editada INTEGER DEFAULT 0,
        editada_por TEXT,
        editada_en TIMESTAMP,
        FOREIGN KEY (producto_id) REFERENCES productos(id),
        FOREIGN KEY (vendedor_id) REFERENCES usuarios(id)
    )
    """)

cur.execute("""
    CREATE TABLE IF NOT EXISTS actividad (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        usuario_id INTEGER,
        usuario_nombre TEXT NOT NULL,
        accion TEXT NOT NULL,
        descripcion TEXT,
        ip TEXT,
        fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
    )
    """)

conn.commit()
print("✅ Base de datos inicializada correctamente.")
except sqlite3.Error as e:
    conn.rollback()
    print(f"❌ Error al inicializar la base de datos: {e}")
finally:
    conn.close()

```

```

def seed_data():
    """Genera registros iniciales ficticios de prueba en el sistema."""
    conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute("SELECT COUNT(*) FROM usuarios")
        if cur.fetchone()[0] > 0:
            return

        usuarios_seed = [
            ("Administrador", "admin", generate_password_hash("admin123"),
"admin", "👑"),
            ("Carlos Vendedor", "vendedor1", generate_password_hash("venta123"),
"vendedor", "👤"),
            ("María Vendedora", "vendedor2", generate_password_hash("venta456"),
"vendedor", "👩"),
        ]
        cur.executemany(
            "INSERT INTO usuarios (nombre, username, password_hash, rol,
avatar_emoji) VALUES (?, ?, ?, ?, ?)",
            usuarios_seed
        )

        productos_seed = [
            ("Laptop", "Electrónica", 18500.0, "💻", 15, 3),
            ("Audífonos", "Audio", 850.0, "🎧", 42, 5),
            ("Mouse", "Accesorios", 320.0, "🖱️", 38, 5),
            ("Teclado", "Accesorios", 580.0, "⌨️", 25, 5),
            ("Monitor", "Electrónica", 4200.0, "🖥️", 8, 3),
            ("Cámara", "Electrónica", 6800.0, "📷", 5, 3),
            ("Tablet", "Electrónica", 7500.0, "📱", 12, 3),
            ("Bocina", "Audio", 1200.0, "🔊", 20, 5),
            ("USB 64GB", "Almacenamiento", 180.0, "💾", 60, 10),
            ("Mochila Tech", "Accesorios", 650.0, "🎒", 30, 5),
            ("Webcam", "Electrónica", 950.0, "📹", 18, 5),
            ("Cable HDMI", "Accesorios", 120.0, "🔌", 75, 10),
        ]
        cur.executemany(
            "INSERT INTO productos (nombre, categoria, precio_base, emoji,
stock, stock_minimo) VALUES (?, ?, ?, ?, ?, ?)",
            productos_seed
        )

        cur.execute("SELECT id, nombre, precio_base, categoria, emoji FROM
productos")
        productos = cur.fetchall()
        cur.execute("SELECT id, nombre FROM usuarios WHERE rol='vendedor'")
        vendedores = cur.fetchall()

        ahora = datetime.now()

```



```

        random.seed(42)

        for i in range(20):
            prod = random.choice(productos)
            vend = random.choice(vendedores)
            cant = random.randint(1, 3)
            precio = round(prod["precio_base"] * random.uniform(0.98, 1.02), 2)
            total = round(precio * cant, 2)
            fecha = ahora - timedelta(days=random.randint(1, 28))

            cur.execute(
                """INSERT INTO ventas
                    (producto_id, producto_nombre, categoria, cantidad,
precio_unitario, total, vendedor_id, vendedor_nombre, fecha)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                (prod["id"], prod["nombre"], prod["categoria"], cant, precio,
total, vend["id"], vend["nombre"], fecha.strftime("%Y-%m-%d %H:%M:%S"))
            )

            conn.commit()
        finally:
            conn.close()

```

```

def registrar_actividad(conn, usuario_id, usuario_nombre, accion, descripcion,
ip=""):
    conn.execute(
        "INSERT INTO actividad (usuario_id, usuario_nombre, accion, descripcion,
ip) VALUES (?, ?, ?, ?, ?)",
        (usuario_id, usuario_nombre, accion, descripcion, ip)
    )

```

```

def get_dashboard_metrics(user_id, rol):
    """Consulta agregados SQL para poblar el Dashboard analítico principal."""
    conn = get_connection()
    try:
        hoy = datetime.now().strftime("%Y-%m-%d")
        mes_inicio = datetime.now().strftime("%Y-%m-01")

        filtro_vendedor = "" if rol == "admin" else f"AND vendedor_id =
{user_id}"

        row = conn.execute(
            f"SELECT COUNT(*) as cnt, COALESCE(SUM(total),0) as suma FROM ventas
"
            f"WHERE DATE(fecha)=? AND estado='completada' {filtro_vendedor}",
            (hoy,)
        ).fetchone()
        ventas_hoy = row["cnt"]
        ingresos_hoy = row["suma"]

```

```

        row = conn.execute(
            f"SELECT COALESCE(SUM(total),0) as suma FROM ventas "
            f"WHERE DATE(fecha)>=? AND estado='completada' {filtro_vendedor}",
            (mes_inicio,)
        ).fetchone()
        ingresos_mes = row["suma"]

        row = conn.execute(
            f"SELECT COALESCE(AVG(total),0) as avg FROM ventas "
            f"WHERE DATE(fecha)>=? AND estado='completada' {filtro_vendedor}",
            (mes_inicio,)
        ).fetchone()
        venta_promedio = row["avg"]

        row = conn.execute(
            f"SELECT producto_nombre, SUM(total) as rev FROM ventas "
            f"WHERE DATE(fecha)>=? AND estado='completada' {filtro_vendedor} "
            f"GROUP BY producto_nombre ORDER BY rev DESC LIMIT 1", (mes_inicio,)
        ).fetchone()
        mejor_producto = dict(row) if row else {"producto_nombre": "-", "rev":
0}

        stock_bajo = conn.execute(
            "SELECT id, nombre, emoji, stock, stock_minimo FROM productos "
            "WHERE stock <= stock_minimo AND activo=1 ORDER BY stock ASC"
        ).fetchall()

        ultimas_ventas = conn.execute(
            f"SELECT v.*, p.emoji FROM ventas v "
            f"LEFT JOIN productos p ON v.producto_id=p.id "
            f"WHERE 1=1 {filtro_vendedor} ORDER BY v.fecha DESC LIMIT 8"
        ).fetchall()

        return {
            "ventas_hoy":      ventas_hoy,
            "ingresos_hoy":    round(ingresos_hoy, 2),
            "ingresos_mes":    round(ingresos_mes, 2),
            "venta_promedio":  round(venta_promedio, 2),
            "mejor_producto":  mejor_producto,
            "stock_bajo":      [dict(r) for r in stock_bajo],
            "ultimas_ventas":  [dict(r) for r in ultimas_ventas],
        }
    finally:
        conn.close()

```

6. MATRIZ DE CASOS DE PRUEBA (QA)

Plan formal de pruebas de caja negra ejecutado sobre los flujos interactivos del sistema:

Módulo de Pruebas 1: Autenticación de Usuarios

ID	Entrada de Datos	Comportamiento Esperado	Estatus
1.1	Username: "admin", Password: "admin123"	Login exitoso, inicialización de cookie y redirección forzada a /dashboard.	✓ PASÓ
1.2	Username: "vendedor1", Password: "venta123"	Acceso aprobado. Contexto limitado a vistas operativas.	✓ PASÓ
1.3	Username: "admin", Password: "incorrecta"	Rechazo de login, retorno de código 412/401 y renderizado del error.	✓ PASÓ
1.4	Username: "noexiste", Password: "cualquier"	Error de credenciales genérico (prevención de enumeración de usuarios). Log de auditoría grabado.	✓ PASÓ
1.5	Estructura JSON con campos vacíos	Bloqueo inmediato por validador interno. Retorno HTTP 400.	✓ PASÓ

Módulo de Pruebas 2: Registro de Nueva Venta

ID	Entrada de Datos	Comportamiento Esperado	Estatus
2.1	ID Producto: Laptop (Stock 15), Cantidad: 2, Precio: 18500	Registro exitoso. Nuevo stock fijado en 13 de forma atómica en BD.	✓ PASÓ
2.2	ID Producto: Mouse (Stock 38), Cantidad: 5, Precio: 320	Cómputo correcto de total (\$1,600.00) y persistencia autorizada.	✓ PASÓ
2.3	ID Producto: USB 64GB (Stock 60), Cantidad: 70	Bloqueo lof transaccional. Retorno de mensaje descriptivo de stock insuficiente.	✓ PASÓ
2.4	Cantidad de compra: 0	Fallo por violación de restricción en validación backend. HTTP 400.	✓ PASÓ
2.5	Precio pactado negativo: -100	Rechazo inmediato. Evita inyección de saldos negativos a favor.	✓ PASÓ

Módulo de Pruebas 3: Control de Acceso y Reglas de Seguridad

ID	Acción Solicitada	Comportamiento Esperado	Estatus
3.1	Petición GET directa a /dashboard desde cliente anónimo	Interceptado por el decorador @login_required; redirección automática a /login.	✓ PASÓ
3.2	Intento de guardado de producto (API POST) por rol 'vendedor'	Interceptado por el decorador de privilegios; bloqueo con código HTTP 403.	✓ PASÓ
3.3	Vendedor intenta editar una transacción que no le pertenece	Evaluación en servidor de la tupla vendedor_id == current_user.id; denegación con código 403.	✓ PASÓ

8. CONCLUSIONES

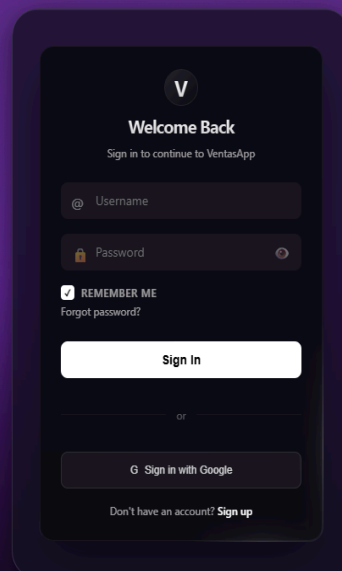
El desarrollo estructurado de **VentasApp** marca una transición crítica desde la resolución guiada de algoritmos básicos hacia el diseño estratégico de soluciones listas para su despliegue comercial real. La integración coordinada de bases de datos relacionales, manejo estricto de sesiones HTTP y middleware de seguridad consolida las competencias clave requeridas en la Ingeniería de Software profesional.

8. Evidencia Visual

INICIALIZACIÓN DEL SCRIPT

```
(.venv) PS C:\Users\aklla\OneDrive\Escritorio\sales_system> & c:\Users\aklla\OneDrive\Escritorio\sales_system\.venv\Scripts\python.exe
[✓] Base de datos inicializada correctamente.
[ℹ] La base de datos ya contiene datos. Omitiendo seed.
=====
🖨 VentasApp – Sistema de Registro de Ventas
=====
🌐 URL: http://localhost:5000
👑 Admin: admin / admin123
👤 Vendedor: vendedor1 / venta123
👤 Vendedor: vendedor2 / venta456
=====
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
[✓] Base de datos inicializada correctamente.
[ℹ] La base de datos ya contiene datos. Omitiendo seed.
=====
🖨 VentasApp – Sistema de Registro de Ventas
=====
🌐 URL: http://localhost:5000
👑 Admin: admin / admin123
👤 Vendedor: vendedor1 / venta123
👤 Vendedor: vendedor2 / venta456
=====
* Debugger is active!
* Debugger PIN: 743-393-529
```

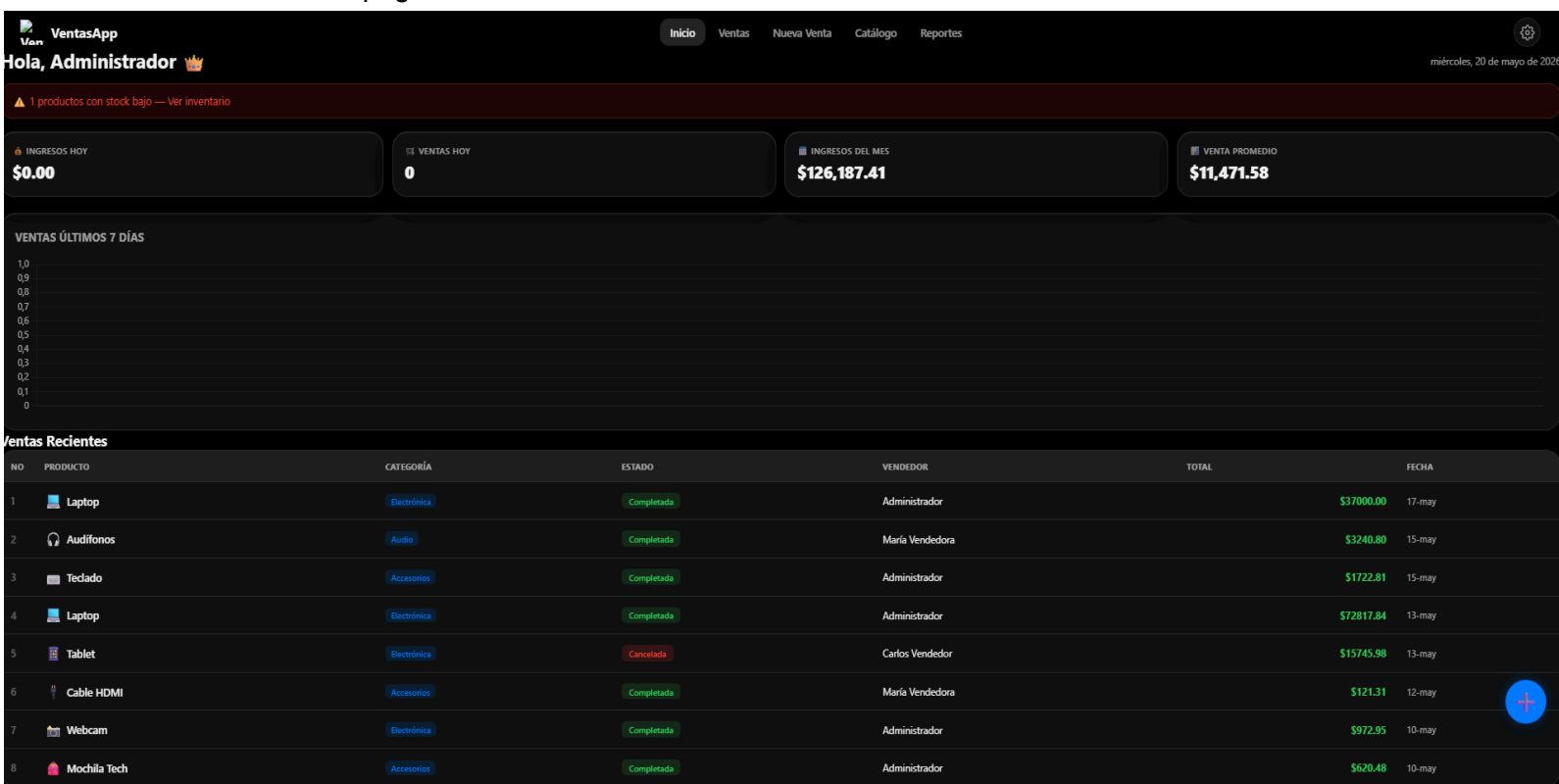
Pantalla de login- simplemente es un login que se valida que el usuario sea el correcto y la contraseña sean las correctas



Despues sigue la pagina principal de la app
lo primero que se ve es el header de arriba con las opciones:

Inicio (el dashboard)
Ventas (historial de todas)
Nueva Venta (donde estás ahora)
Catálogo (ver productos)
Reportes (ver gráficas)

Ese botón "+" que está ahí te lleva a esta sección de Nueva Venta. Es un atajo rápido si estás en otra página.



Seccion de Nueva Venta - Aqui se tienen que llenar los datos de las ventas en los siguientes campos

PRODUCTO Buscas qué vendiste (laptop, mouse, teclado, etc.)

CANTIDAD Cuántas unidades vendiste (ej: 2 laptops)

PRECIO UNITARIO Cuánto cuesta cada una (ej: \$18,500 por laptop)

NOTAS Datos extra si quieres (como "cliente VIP" o "con descuento")

VENDEDOR Automático - te muestra quién está vendiendo (en este caso "Administrador")

FECHA Automática - la fecha y hora exacta de la venta

El Total

Ahí abajo aparece \$0.00 (o el monto que calcule) - es básicamente:

Conforme completas los campos, se actualiza solo.

El Botón Azul

"Registrar Venta" - es el botón que aprietas cuando ya completaste todo. Al hacerlo:

Se guarda la venta en la BD

Se reduce el stock del producto automáticamente

Se registra quién vendió, cuándo, etc.

Nueva Venta 

PRODUCTO *

CANTIDAD *

PRECIO UNITARIO (\$) *

NOTAS (OPCIONAL)

Total a Registrar: **\$0.00**

Registrar Venta 

Nueva Venta 

PRODUCTO *

 **Audifonos**
Audio
\$850.00
Stock: 42

 **Bocina**
Audio
\$1200.00
Stock: 20

 **Cable HDMI**
Accesorios
\$120.00
Stock: 75

 **Cámara**

Total a Registrar: **\$0.00**

Registrar Venta 

Nueva Venta 

PRODUCTO *

CANTIDAD *

PRECIO UNITARIO (\$) *

NOTAS (OPCIONAL)

Total a Registrar: **\$2,400.00**

Registrar Venta 

Después sigue la sección de Ventas que es el registro de todas las ventas que se han registrado en la app
sirve con un filtro de búsqueda y las ventas tienen los siguientes elementos;
Nombre del producto
Categoría
Estado
Vendedor
Total
Fecha
Acciones (Imprimir ticket, editar y borrar)

Historial de Ventas

mochila

Fecha: Fecha

Estado: Estado

Orden: Más reciente

NO	PRODUCTO	CATEGORÍA	ESTADO	VENDEDOR	TOTAL	FECHA	ACCIONES
10	Mochila Tech	Accesorios	completada	Administrador	\$620.48	10 may 2026, 01:53 p.m.	
17	Mochila Tech	Accesorios	completada	María Vendedora	\$1,893.96	02 may 2026, 10:53 a.m.	
8	Mochila Tech	Accesorios	completada	Carlos Vendedor	\$664.93	30 abr 2026, 07:53 p.m.	

Despues sigue el catalogo donde estan los productos en venta y que seran registrados despues de su venta

Catálogo



Audífonos

Audio

\$850.00

Editar



Boxina

Audio

\$1,200.00

Editar



Cable HDMI

Accesorios

\$120.00

Editar



Cámara

Electrónica

\$6,800.00

Editar



IPHONE 17

Smart

\$31,500.00

Editar



Laptop

Electrónica

\$18,500.00

Editar



Mochila Tech

Accesorios

\$650.00

Editar



Monitor

Electrónica

\$4,200.00

Editar



Mouse

Accesorios

\$320.00

Editar

El boton de “+” sirve para agregar mas productos y estos se puedan registrar en la base de datos

Nuevo Producto

EMOJI (CLICK PARA CAMBIAR)

CATEGORÍA

Selecciona...

NOMBRE

PRECIO BASE (\$)

STOCK MÍNIMO (ALERTA)

5

Guardar Producto

CATEGORÍA

Selecciona...

Selecciona...

Bebidas

Snacks

Lácteos

Dulces

Abarrotes

Limpieza

Otros

Inicio

Ventas

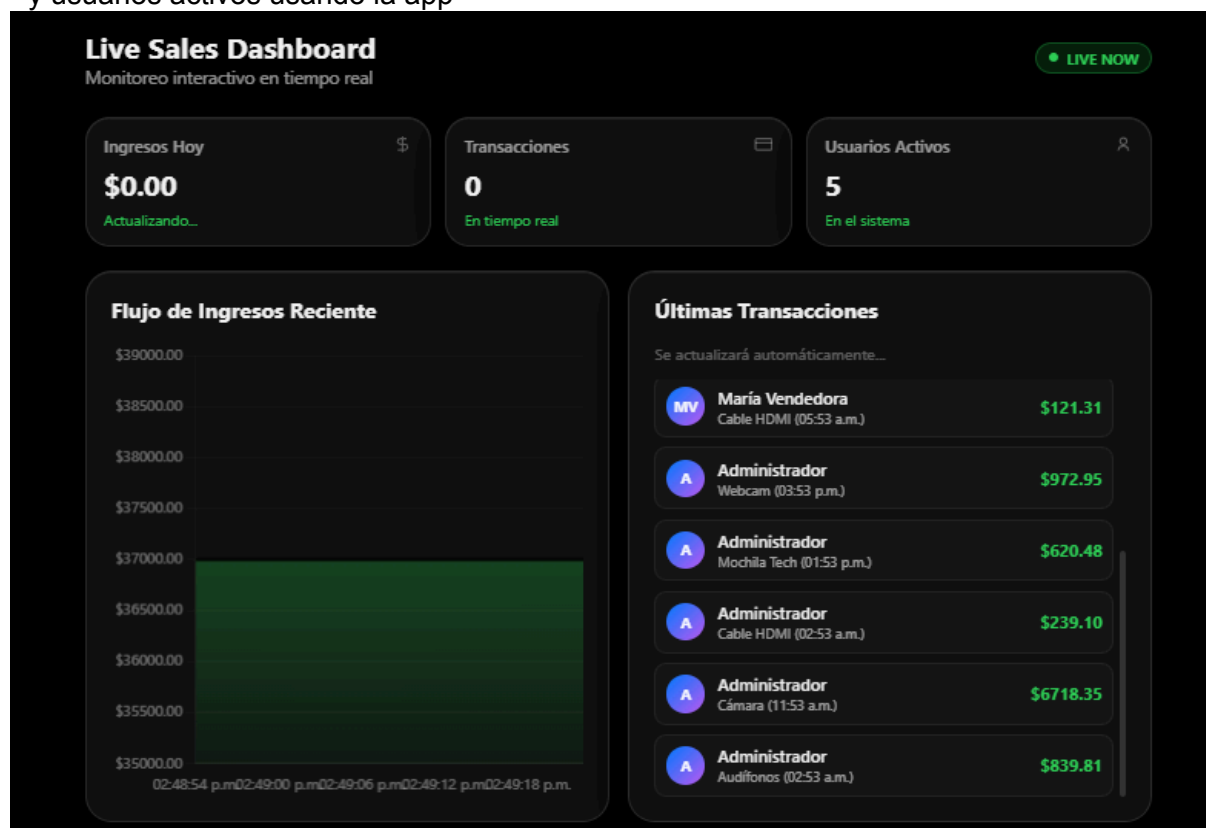
Nueva Venta

Catalogo

Producto guardado ✓

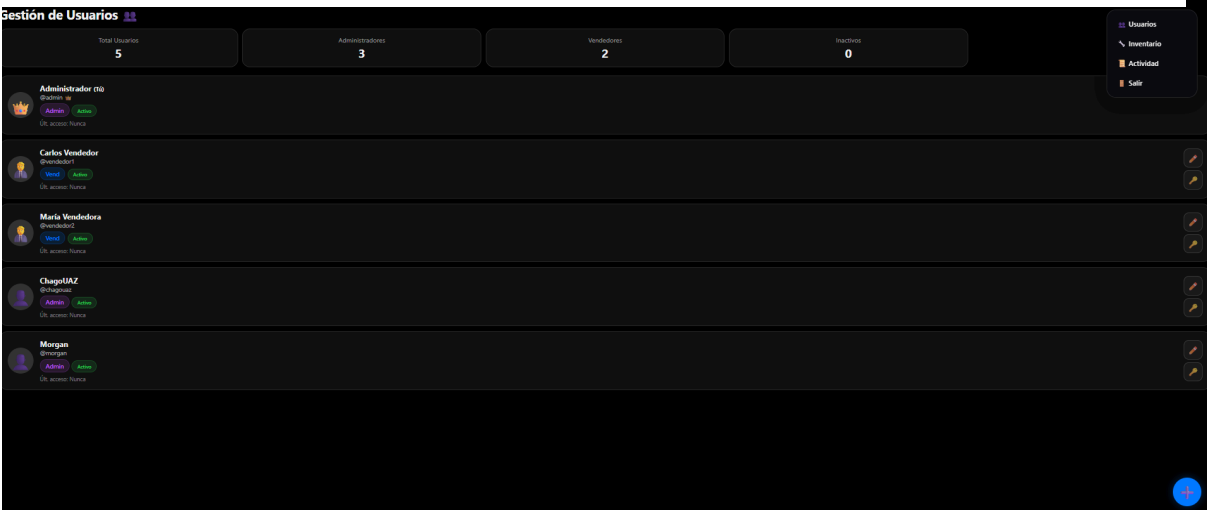
Y por último la parte de Reportes

Donde sale el flujo de ingresos en una grafica, también los ingresos totales , transacciones, y usuarios activos usando la app



Luego en la parte exclusivamente para admin tambien hay un botón de cconfiguraciones

En usuarios es una sección que permite agregar, editar usuarios y resetear contraseñas



Editar Usuario

NOMBRE COMPLETO

Carlos Vendedor

USUARIO (LOGIN)

vendedor1

ROL

Vendedor

ACTIVO

☐

Guardar Usuario

Resetear Contraseña

Se asignará una nueva contraseña para: **Carlos Vendedor**

NUEVA CONTRASEÑA

Cambiar Contraseña

Despues la seccion de inventario que permite gestionar el inventario y controlar stock

PRODUCTO	CAT.	STOCK	MÍNIMO	ESTADO	ACCIONES
IPHONE 17	Otros	0	2	0	Ajustar +/-
Foco	Abarriles	0	5	0	Ajustar +/-
Cámara	Electrónica	5	3	5	Ajustar +/-
Monitor	Electrónica	8	3	8	Ajustar +/-
Tablet	Electrónica	12	3	12	Ajustar +/-
Laptop	Electrónica	13	3	13	Ajustar +/-
Webcam	Electrónica	18	5	18	Ajustar +/-
Bocina	Audio	20	5	20	Ajustar +/-
Teclado	Accesorios	25	5	25	Ajustar +/-
Mochila Tech	Accesorios	30	5	30	Ajustar +/-
Mouse	Accesorios	38	5	38	Ajustar +/-
Audífonos	Audio	42	5	42	Ajustar +/-
USB 64GB	Almacenamiento	60	10	60	Ajustar +/-

Tambien hay una seccion de Actividad que es un registro de logs para ver movimientos que hacen los usuarios tanto admins como vendedores

Registro de Actividad 		Auto-refresh cada 30s
Todos los usuarios ▼ Todas las acciones ▼ Hoy Semana Mes Todas		
 Administrador [new]	Nuevo producto creado: Foco (ID:14)	IP: N/A
 Administrador [edit]	Venta #21 editada por Administrador	
 Administrador [edit]	Venta #21 editada por Administrador	
 Administrador [login]	Inicio de sesión exitoso desde 127.0.0.1	
 Administrador [login]	Inicio de sesión exitoso desde 127.0.0.1	

8. CONCLUSIONES

En conclusión en el desarrollo de ventas app se ha aprendido el manejo de base de datos y su enorme importancia en aplicaciones como está de gestión de productos o simplemente gestion de informacion, una de las dificultades a sido la parte del backend en cómo hacer llamadas dentro de la interfaz gráfica que respondan en tiempo real con las bases de datos y las acciones que debería de hacer, la importancia en la programación sobre mi carrera en ingeniería de software creo que ahora mismo en la situación actual en la que se encuentra este mundo ya no veo tan indispensable el saber programar pero para mi algo mas importante que si es obligatorio y dispensable es conocer cómo funciona el codigo y entenderlo “De nada sirve un carro, si se va a intentar volar ” y el uso de nuevas tecnologías creo que abre un mundo nuevo esto de la ia dicen que nos va a quitar el trabajo a los ingenieros de software pero creo y espero que la ia abra nuevas puertas a nuevos trabajos

8. CONCLUSIONES

Aquí el código deja de ser tarea y empieza a ser **arquitectura**.

El código en funcionamiento se encuentra en